

Sistema de Gestao de Pedidos de E-commerce

Projeto de Disciplina — Tecnologia .NET

Aluno:	Gabriel Bodenmuller
Disciplina:	Tecnologia .NET
Arquivo:	GabrielBodenmuller_TecnologiaNET_pd.pdf
Data:	Junho de 2026
Tecnologia:	.NET 10.0, C# 14, xUnit 2.7, Moq 4.20, coverlet 6.0

Resumo: Este projeto implementa um sistema de gestao de pedidos para um e-commerce brasileiro, aplicando Domain-Driven Design (DDD) com quatro Bounded Contexts, codigo C# com Orientacao a Objetos, principios SOLID e GRASP, e testes unitarios com cobertura superior a 80% do dominio.

Repositorio: <https://github.com/gabrielbller/OrderFlow>

Mapa de Rubricas

A tabela abaixo indexa cada item de rubrica ao local do projeto onde e demonstrado.

#	Rubrica	Demonstrado em
R1	OO: Encapsulamento, Abstracao, Heranca, Polimorfismo	Sec. 2 — Entity, AggregateRoot, ValueObject, Order
R2	OO: Modificadores de acesso, propriedades, metodos, construtores	Sec. 2 — todas as classes do dominio
R3	OO: Heranca e polimorfismo em hierarquias	Sec. 2.1 — hierarquia Entity > AggregateRoot > Order/Customer/Product
R4	OO: Abstracao e encapsulamento	Sec. 2.2 — GetEqualityComponents(), setters privados, Total calculado
R5	DDD: Ubiquitous Language, Entities, VOs, Repositories	Sec. 1.2, 1.4 — Glossario e diagrama visual do dominio
R6	DDD: Aggregate, Bounded Contexts, Domain Services	Sec. 1.3 — diagrama Context Map e tabela de componentes
R7	DDD: Domain Services vs Factories	Sec. 1.5 — tabela comparativa e codigo anotado
R8	DDD: ACL e Context Map	Sec. 1.3, 1.6 — diagrama Context Map + PaymentServiceAdapter
R9	SOLID: Coesao, responsabilidade, baixo acoplamento	Sec. 3 — SRP, OCP, DIP, Low Coupling, High Cohesion com codigo
R10	SOLID: SRP — Single Responsibility Principle	Sec. 3.1 — OrderFactory, Money, PaymentServiceAdapter
R11	GRASP: Low Coupling	Sec. 3.4 — IOrderRepository, referencia por ID entre BCs
R12	GRASP: Controller	Sec. 3.6 — OrderApplicationService orquestra sem logica de negocio
R13	Testes: Principios F.I.R.S.T	Sec. 4.1 — tabela F.I.R.S.T com aplicacao pratica
R14	Testes: Cobertura de regras de negocio	Sec. 4.2 — 72 cenarios documentados (118 execucoes xUnit)
R15	Testes: Mocks e stubs com Moq	Sec. 4.3 — OrderDomainService, OrderFactory e InventoryDomainService com Mock e Verify()
R16	Testes: Cobertura > 80%	Sec. 4.4 — relatorio: 89,3% de cobertura de linhas

1. Modelagem com Domain-Driven Design (DDD)

[Rubricas R5, R6, R7, R8] Esta secao demonstra: Ubiquitous Language, Entities, VOs, Repositories (R5); Aggregate, Bounded Contexts, Domain Services (R6); distincao Domain Services vs Factories (R7); integracao via ACL e Context Map (R8).

1.1 Descricao do Projeto

Problema: Sistemas de e-commerce frequentemente concentram toda a logica em uma unica camada de servico, gerando codigo dificil de manter e testar. Regras como "pedido nao pode ser pago sem estoque" ficam espalhadas sem consistencia.

Solucao: Aplicar DDD para organizar a complexidade em Bounded Contexts bem delimitados, com Aggregates que garantem invariantes e Domain Services que coordenam operacoes entre contextos.

Escopo do Projeto (Interno x Externo)

Interno a aplicacao (modelado no dominio):

- Cadastro e validacao de clientes (idade minima, CPF, e-mail)
- Catalogo de produtos e controle de estoque (reserva e liberacao)
- Ciclo de vida do pedido: criacao, confirmacao, pagamento, envio, entrega e cancelamento
- Regras de negocio e invariantes: calculo de totais, transicoes de status e disponibilidade

Externo a aplicacao (fora dos limites do dominio):

- Gateway de pagamento (ex.: Stripe / PagSeguro) — acessado via Anti-Corruption Layer
- Interface de usuario (web/mobile) — consome a camada de Aplicacao
- Persistencia em banco de dados — abstraída pelos Repositories (neste projeto, InMemory)
- Servico de logistica/transportadora — responsavel pelo envio fisico

Usuarios do Sistema (Atores)

- **Cliente:** pessoa que se cadastra e realiza pedidos (atraves da UI que consome a aplicacao).
- **Operador / Atendente:** gerencia o catalogo de produtos, o estoque e acompanha os pedidos.
- **Gateway de Pagamento (sistema externo):** processa cobranças e estornos, integrado via Anti-Corruption Layer.
- **Servico de Logistica (sistema externo):** recebe os pedidos pagos para realizar o envio.

Principais Regras de Negocio

1. Cliente deve ter no minimo 18 anos para se cadastrar
2. Pedido so pode ser confirmado se todos os itens tiverem estoque disponivel
3. Fluxo de status: Pending > Confirmed > Paid > Shipped > Delivered
4. Cancelamento permitido em Pending, Confirmed ou Paid (pago libera o estoque reservado)
5. Pedido sem itens nao pode ser confirmado
6. CPF deve ser valido (algoritmo de digitos verificadores)
7. Valores monetarios sao positivos e arredondados em 2 casas decimais

1.2 Linguagem Ubiqua (Ubiquitous Language)

[Rubrica R5] Termos do dominio compartilhados entre desenvolvedores e especialistas de negocio. A Ubiquitous Language garante que o codigo reflita o vocabulario do dominio.

Termo (Codigo)	Significado no dominio
Order	Solicitacao formal de compra feita por um Cliente
OrderItem	Produto + Quantidade + Preco Unitario dentro de um Pedido
Customer	Pessoa fisica com CPF, e-mail e data de nascimento validos
Product	Mercadoria disponivel para venda, com SKU (ProductCode) e estoque
Money	Par (valor decimal, codigo ISO de moeda) — sempre positivo e imutavel
OrderStatus	Estado atual no ciclo de vida: Pending, Confirmed, Paid, Shipped, Delivered, Cancelled
OrderPlacedEvent	Evento de dominio disparado quando um pedido e confirmado
PaymentResult	Resultado de uma cobranca (sucesso/falha) — linguagem do dominio, nao do gateway
IOrderRepository	Contrato de persistencia para o Aggregate Order (DIP)

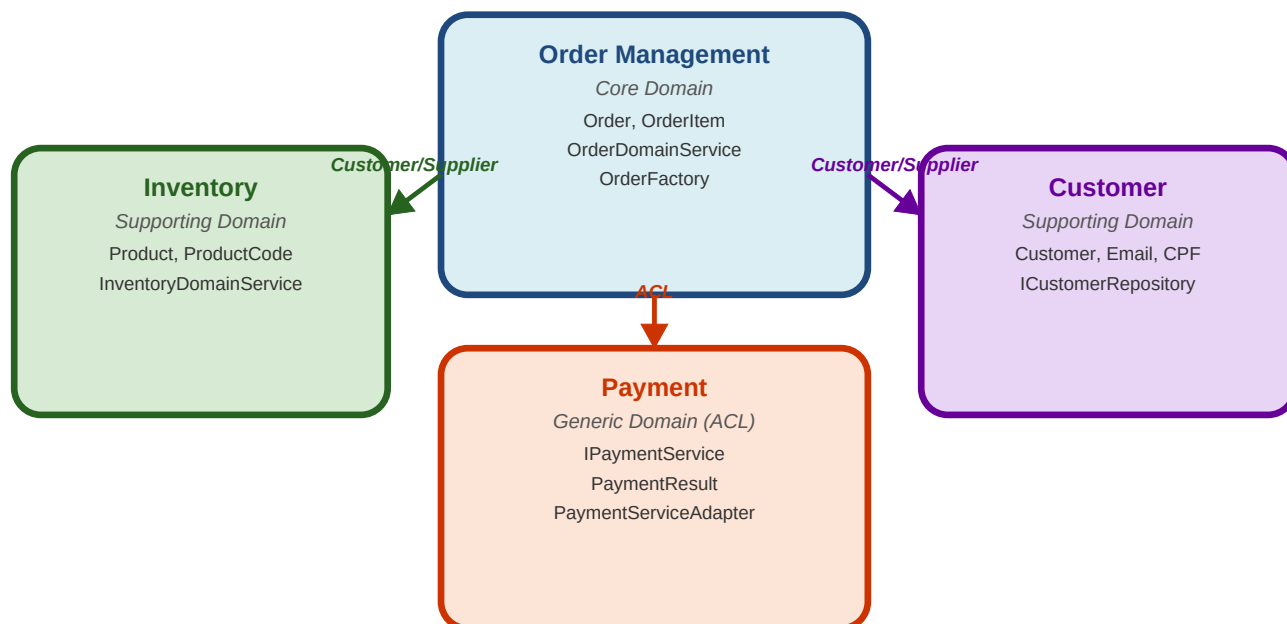
1.3 Bounded Contexts e Context Map

[Rubricas R6, R8] Quatro Bounded Contexts identificados, cada um com modelo e linguagem propios. O Context Map define como os contextos se integram.

Diagrama Visual — Context Map:

Context Map — Integracao entre Bounded Contexts

ACL = Anti-Corruption Layer: PaymentServiceAdapter traduz entre o dominio e o gateway externo



Bounded Context	Tipo	Principais Componentes	Relacao no Context Map
Order Management	Core Domain	Order, OrderItem OrderDomainService, OrderFactory	Downstream — consome Inventory, Customer e Payment
Inventory	Supporting Domain	Product, ProductCode InventoryDomainService	Upstream (Supplier) de Order Mgmt
Customer	Supporting Domain	Customer, Email, CPF ICustomerRepository	Upstream (Supplier) de Order Mgmt
Payment	Generic Domain	IPaymentService (ACL) PaymentResult	ACL — isolado por adapter

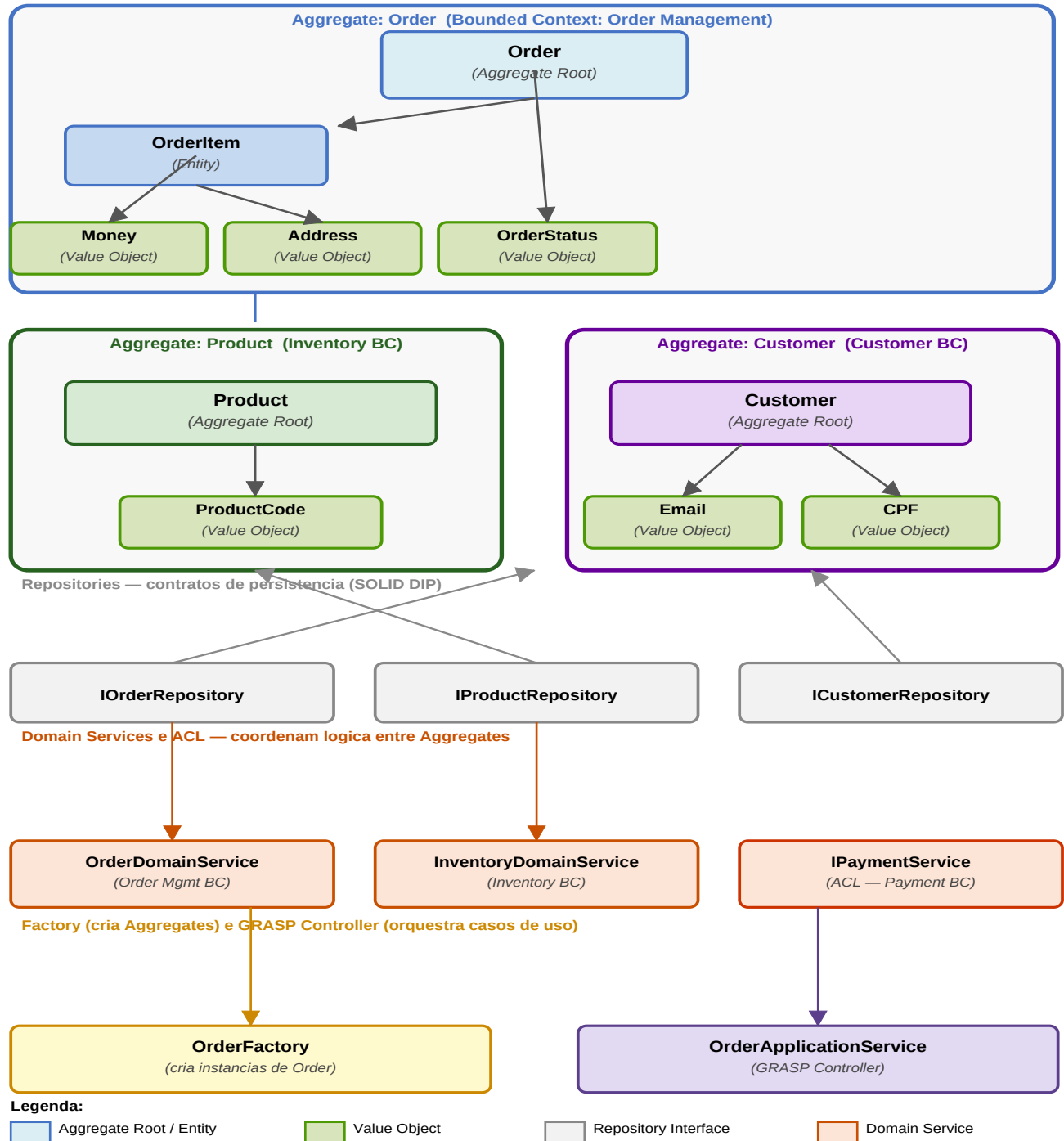
Anti-Corruption Layer (ACL) — Rubrica R8

A ACL protege o dominio de mudancas na API do gateway de pagamento externo. A interface `IPaymentService` pertence ao dominio. O `PaymentServiceAdapter` (infraestrutura) traduz entre a linguagem do dominio e a API externa (que usa centavos inteiros, strings de status, etc.). Se o gateway mudar, apenas o adapter muda — o dominio permanece inalterado.

1.4 Diagrama de Dominio — Aggregates, Entities e Value Objects

[Rubrica R5, R6] Diagrama visual do modelo de dominio mostrando os Aggregates, Entities, Value Objects, Repositories, Domain Services e Factory.

Diagrama Visual — Modelo de Dominio:



Elemento DDD	Tipo	Bounded Context	Responsabilidade
Order	Aggregate Root	Order Management	Gerencia ciclo de vida e invariantes do pedido

Elemento DDD	Tipo	Bounded Context	Responsabilidade
OrderItem	Entity (interna)	Order Management	Produto + quantidade + preco no pedido
Money	Value Object	Order Management	Valor monetario imutavel com operacoes aritmeticas
Address	Value Object	Order Management	Endereco de entrega validado e imutavel
OrderStatus	Value Object	Order Management	Estado com regras de transicao encapsuladas
Product	Aggregate Root	Inventory	Catalogo e controle de estoque de produtos
ProductCode	Value Object	Inventory	Codigo SKU validado (4-12 chars alfanumericos)
Customer	Aggregate Root	Customer	Dados com validacao de CPF, email e idade >= 18
Email	Value Object	Customer	Email validado por regex e normalizado para lowercase
CPF	Value Object	Customer	CPF com validacao de digitos verificadores
PaymentResult	Value Object	Payment	Resultado de cobranca na linguagem do dominio
IOrderRepository	Repository Interface	Order Management	Contrato de persistencia — DIP
OrderDomainService	Domain Service	Order Management	Coordena confirmacao e pagamento entre Aggregates
InventoryDomainService	Domain Service	Inventory	Disponibilidade e reserva de estoque
OrderFactory	Factory	Order Management	Cria instancias de Order com validacao de pre-condicoes
OrderApplicationService	GRASP Controller	Application	Coordena casos de uso sem conter logica de negocio

1.5 Domain Services vs Factories

[Rubrica R7] Distincao clara entre os dois padroes DDD com base na responsabilidade.

Criterio	Domain Service	Factory
Responsabilidade	Coordenar operacoes de negocio entre Aggregates	Criar instancias complexas de Aggregates
Contem regras de negocio?	SIM — verificacao de estoque, fluxo de pagamento	NAO — apenas valida pre-condicoes de criacao
Retorna?	Efeito colateral (altera estado dos Aggregates)	Nova instancia do Aggregate recém-criado
Exemplo no projeto	<code>OrderDomainService.ConfirmOrderAsync()</code>	<code>OrderFactory.CreateAsync()</code>
Dependencias	Repositorios + Domain Services + <code>IPaymentService</code>	Repositorios apenas para validar existencia

Listagem: Comparacao: Domain Service (coordena negocio) vs Factory (cria Aggregate)

```
// ---- OrderDomainService.cs – Domain Service ----
// Contem REGRAS DE NEGOCIO e coordena entre Aggregates
public async Task ConfirmOrderAsync(Order order, ...)
{
    foreach (var item in order.Items)
    {
        // Regra de negocio: verificar estoque (nao pertence a Order nem a Product isolados)
        var ok = await _inventoryService.IsStockAvailableAsync(item.ProductId, item.Quantity);
        if (!ok) throw new InvalidOperationException("Estoque insuficiente.");
    }
    order.Confirm(); // altera estado do Aggregate
    await _orderRepo.UpdateAsync(order); // persiste via repositório
}

// ---- OrderFactory.cs – Factory (SEM regras de negocio, apenas CRIA) ----
public async Task<Order> CreateAsync(Guid customerId, Address addr, ...)
{
    var customer = await _customerRepo.GetByIdAsync(customerId)
    ?? throw new InvalidOperationException("Cliente não encontrado.");
    if (!customer.IsActive) throw new InvalidOperationException("Cliente inativo.");

    var order = new Order(Guid.NewGuid(), customerId, addr); // CRIA o Aggregate
    foreach (var (pid, qty) in items)
    {
        var product = await _productRepo.GetByIdAsync(pid);
        order.AddItem(product.Id, product.Name, qty, product.Price);
    }
    return order; // retorna o Aggregate pronto – não persiste, não executa negocio
}
```

2. Orientacao a Objetos com C#

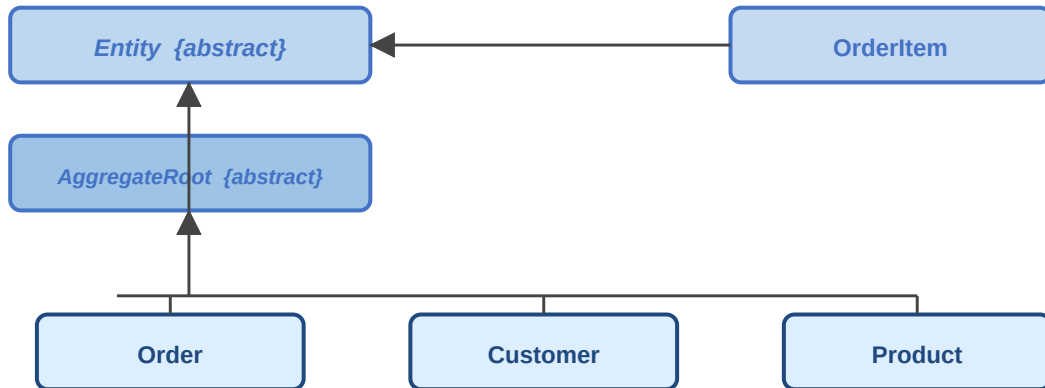
[Rubricas R1, R2, R3, R4] Encapsulamento, Abstracao, Heranca e Polimorfismo (R1); modificadores de acesso, propriedades, metodos e construtores (R2); hierarquias flexiveis (R3); abstracao e encapsulamento (R4).

2.1 Hierarquia de Classes — Heranca e Polimorfismo

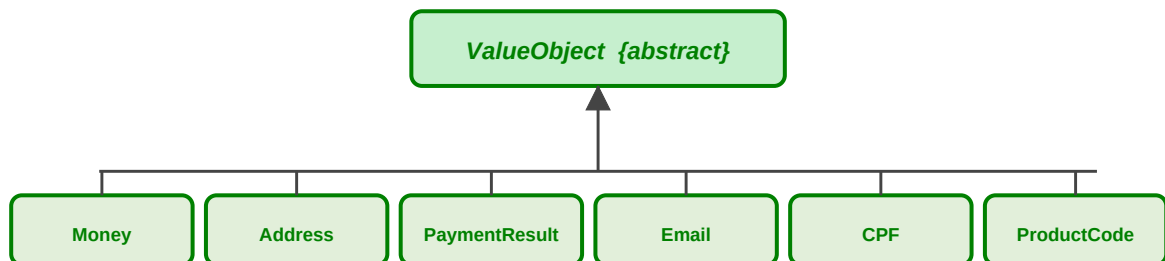
[Rubrica R3] Hierarquia de 3 niveis para Entities: Entity {abstract} > AggregateRoot {abstract} > classes concretas. Value Objects herdam de ValueObject {abstract}.

Diagrama Visual — Hierarquia de Heranca:

Entidades — herdam de Entity / AggregateRoot



Value Objects — herdam de ValueObject



Classe Base	Herda de	Classes Concretas que herdam
Entity {abstract}	(raiz)	OrderItem — herda Entity diretamente
AggregateRoot {abstract}	Entity	Order, Customer, Product
ValueObject {abstract}	(raiz)	Money, Address, PaymentResult, Email, CPF, ProductCode
DomainEvent {abstract}	(raiz)	OrderPlacedEvent, OrderCancelledEvent

Nota: OrderStatus tambem e um Value Object conceitual (ver Secao 1.4), porem implementado como um tipo-seguro de estado (state) com igualdade propria, sem herdar a base ValueObject — por isso nao aparece nesta hierarquia de heranca.

Listagem: Heranca e Polimorfismo — Entity, AggregateRoot, ValueObject

```

// 00 - HERANCA (3 niveis): Entity → AggregateRoot → Order
public abstract class Entity { public Guid Id { get; protected set; } }
public abstract class AggregateRoot : Entity // herda Id de Entity
{
    protected void AddDomainEvent(DomainEvent e) { ... }
}
public sealed class Order : AggregateRoot // herda AddDomainEvent e Id
{
    public void Confirm()
    {
        TransitionTo(OrderStatus.Confirmed);
        AddDomainEvent(new OrderPlacedEvent(Id, CustomerId, Total)); // Id herdado!
    }
}
  
```

```
}

// OO - POLIMORFISMO: GetEqualityComponents() diferente em cada Value Object
public abstract class ValueObject
{
    protected abstract IEnumerable<object> GetEqualityComponents(); // ponto polimorf.
    public override bool Equals(object? obj) // algoritmo unico
        => GetEqualityComponents()
            .SequenceEqual(((ValueObject)obj).GetEqualityComponents());
}
public sealed class Money : ValueObject // compara Amount + Currency (2 campos)
{
    protected override IEnumerable<object> GetEqualityComponents()
    { yield return Amount; yield return Currency; }
}
public sealed class Address : ValueObject // compara 8 campos – mesmo algoritmo, outra impl.
{
    protected override IEnumerable<object> GetEqualityComponents()
    { yield return Street; yield return Number; yield return City; /* + 5 */ }
}
```

2.2 Encapsulamento e Abstracao

[Rubricas R1, R2, R4] Setters privados/protected, construtores com validacao, metodos que ocultam detalhes de implementacao. Modificadores: public, private, protected, internal.

Listagem: Encapsulamento e Abstracao — Order, OrderItem, Money

```
// 00 - ENCAPSULAMENTO (Order.cs)
public sealed class Order : AggregateRoot
{
    private readonly List<OrderItem> _items = new(); // private: nunca exposto diretamente
    public OrderStatus Status { get; private set; } // private set: so via metodos
    // ABSTRACAO: Total calculado dinamicamente – implementacao oculta do consumidor
    public Money Total => _items.Aggregate(Money.Zero("BRL"), (a, i) => a.Add(i.Subtotal));
    // Encapsulamento de colecao: externo so le, nunca modifica diretamente
    public IReadOnlyList<OrderItem> Items => _items.AsReadOnly();
    // Metodo PRIVADO – logica de estado protegida de acesso externo
    private void TransitionTo(OrderStatus s)
    {
        if (!Status.CanTransitionTo(s)) throw new InvalidOperationException(...);
        Status = s;
    }
}

// 00 - Modificadores de acesso:
// public → acessivel por qualquer chamador (API de dominio)
// private → apenas esta classe (encapsulamento total)
// protected → subclasses (heranca controlada)
// internal → mesmo assembly (OrderItem – so Order pode instanciar)

public sealed class OrderItem : Entity
{
    internal OrderItem(...) : base(id) { } // internal: nao instanciavel fora de Domain.dll
    internal void UpdateQuantity(int qty) { } // internal: so Order pode chamar
}

// 00 - ABSTRACAO + ENCAPSULAMENTO (Money.cs)
public sealed class Money : ValueObject
{
    public decimal Amount { get; } // { get; } sem set: imutavel apos construcão
    public string Currency { get; } // { get; } sem set: imutavel apos construcão
    public Money(decimal amount, string currency) // construtor valida e normaliza
    {
        if (amount < 0) throw new ArgumentException("Valor nao pode ser negativo.");
        Amount = Math.Round(amount, 2); // regra de negocio encapsulada
        Currency = currency.ToUpperInvariant(); // normalizacao encapsulada
    }
}
```

3. Principios SOLID e GRASP

[Rubricas R9, R10, R11, R12] SOLID com coesao, responsabilidade e baixo acoplamento (R9); SRP (R10); GRASP Low Coupling (R11); GRASP Controller (R12). Para cada principio: nome, objetivo e trecho de codigo explicado.

3.1 SOLID — SRP: Single Responsibility Principle

Principio SRP — Objetivo: Uma classe deve ter apenas UM motivo para mudar. Sem SRP, teriamos um God Service com criacao, validacao, pagamento e persistencia em uma so classe — impossivel de manter e testar.

Listagem: SRP — Cada classe tem um unico motivo para mudar

```
// =====
// SOLID - SRP: CADA CLASSE TEM UM UNICO MOTIVO PARA MUDAR
// =====

// OrderFactory – motivo de mudanca: processo de CRIACAO de Order
// Nao contem regras de fluxo, nao persiste dados
public class OrderFactory
{
    public async Task<Order> CreateAsync(Guid customerId, Address addr, ...) { ... }
}

// OrderDomainService – motivo de mudanca: FLUXO DE NEGOCIO (confirmacao, pagamento)
// Nao cria objetos, nao acessa banco diretamente
public class OrderDomainService
{
    public async Task ConfirmOrderAsync(Order order, ...) { ... }
    public async Task ProcessPaymentAsync(Order order, string token, ...) { ... }
}

// IOrderRepository – motivo de mudanca: CONTRATO DE PERSISTENCIA
// Nao contem logica de negocio
public interface IOrderRepository
{
    Task<Order?> GetByIdAsync(Guid id, ...);
    Task AddAsync(Order order, ...);
    Task UpdateAsync(Order order, ...);
}

// Money – motivo de mudanca: REGRAS DE VALOR MONETARIO
// Nao sabe nada sobre pedidos, clientes ou pagamentos
public sealed class Money : ValueObject
{
    public Money Add(Money other) { ... }
    public Money Subtract(Money other) { ... }
    public Money Multiply(decimal factor) { ... }
}

// PaymentServiceAdapter – motivo de mudanca: TRADUCAO para o gateway externo
// Se o gateway mudar, apenas este arquivo muda. Dominio inalterado.
public class PaymentServiceAdapter : IPaymentService { ... }
```

3.2 SOLID — OCP: Open/Closed Principle

Principio OCP — Objetivo: Classes abertas para extensao, fechadas para modificacao. Novos Value Objects sao criados sem alterar ValueObject. Novos estados de pedido sao adicionados sem alterar Order.

Listagem: OCP — Extensao sem modificacao

```
// SOLID - OCP: ValueObject FECHADA para modificacao, ABERTA para extensao
```

```

public abstract class ValueObject
{
    protected abstract IEnumerable<object> GetEqualityComponents(); // ponto de extensao
    public override bool Equals(object? obj) { ... } // FECHADO — nunca modificado
}

// EXTENSAO sem modificar ValueObject — novo VO criado por heranca:
public sealed class ProductCode : ValueObject // novo VO, ValueObject nao mudou
{
    public string Value { get; }
    public ProductCode(string value) { /* valida formato SKU 4-12 chars */ }
    protected override IEnumerable<object> GetEqualityComponents()
    { yield return Value; }
}

// SOLID - OCP: OrderStatus — novo estado adicionado sem modificar Order.cs
public bool CanTransitionTo(OrderStatus next)
{
    return (this, next) switch
    {
        var (c, n) when c == Pending && n == Confirmed => true,
        var (c, n) when c == Confirmed && n == Paid => true,
        var (c, n) when c == Paid && n == Shipped => true,
        // Para adicionar "ReturnRequested", basta add um case aqui
        // Order.cs nao precisa mudar — OCP
        _ => false
    };
}

```

3.3 SOLID — DIP: Dependency Inversion Principle

Principio DIP — Objetivo: Modulos de alto nivel nao devem depender de modulos de baixo nivel. Ambos devem depender de abstracoes. Todo acoplamento do dominio e via interface — nunca via `new Concreto()`.

Listagem: DIP — Dependencias via interfaces e injecao de construtor

```

// SOLID - DIP: OrderDomainService depende APENAS de interfaces (abstracoes)
public class OrderDomainService
{
    private readonly IOrderRepository _orderRepository; // interface
    private readonly IProductRepository _productRepository; // interface
    private readonly IPaymentService _paymentService; // interface (ACL)
    private readonly InventoryDomainService _inventoryService; // injetado

    // Injecao de dependencia via construtor (nao usa new() de concreto)
    public OrderDomainService(
        IOrderRepository orderRepository, // pode ser InMemory ou EF Core
        IProductRepository productRepository, // qualquer implementacao
        IPaymentService paymentService, // Stripe, PagSeguro — indiferente
        InventoryDomainService inventoryService)
    {
        _orderRepository = orderRepository
            ?? throw new ArgumentNullException(nameof(orderRepository));
        _paymentService = paymentService
            ?? throw new ArgumentNullException(nameof(paymentService));
        // ...
    }
}

// Dominio NUNCA faz: new SqlOrderRepository() ou new StripePaymentService()
// Testavel: mocks podem ser injetados — ver OrderDomainServiceTests.cs

```

3.4 GRASP — Low Coupling

Padrao Low Coupling — Objetivo: Minimizar dependencias entre classes. Mudancas em uma classe nao propagam efeitos em outras. Aplicado via interfaces, referencias por ID entre BCs e separacao em camadas.

Listagem: GRASP Low Coupling — referencias por ID e interfaces

```
// GRASP - Low Coupling: OrderItem nao referencia Product diretamente
// Bounded Contexts comunicam-se por IDs, nao por referencias diretas
public sealed class OrderItem : Entity
{
    public Guid    ProductId    { get; }    // ID – coupling minimo entre BCs
    public string  ProductName { get; }    // copia do nome (historico de compra)
    public Money   UnitPrice    { get; }    // copia do preco (historico de compra)
    // NAO tem: public Product Product { get; }
    // Isso evitaria acoplamento direto entre Order Management e Inventory BCs
}

// GRASP - Low Coupling: IOrderRepository desacopla dominio da infraestrutura
// O dominio declara apenas o CONTRATO necessario, sem saber a implementacao
public interface IOrderRepository // pertence ao DOMINIO
{
    Task<Order?> GetByIdAsync(Guid id, ...);
    Task AddAsync(Order order, ...);
    Task UpdateAsync(Order order, ...);
}

// INFRA implementa sem afetar o dominio:
public class InMemoryOrderRepository : IOrderRepository { ... } // para testes
// Futuramente: public class EfCoreOrderRepository : IOrderRepository { ... }
// Dominio nao muda – apenas a implementacao e substituida
```

3.5 GRASP — High Cohesion

Padrao High Cohesion — Objetivo: Cada classe contem responsabilidades fortemente relacionadas entre si. InventoryDomainService so trata de estoque. Money so trata de valores monetarios.

Listagem: GRASP High Cohesion — responsabilidades coesas

```
// GRASP - High Cohesion: InventoryDomainService
// TODOS os metodos tratam do mesmo tema: ESTOQUE
public class InventoryDomainService
{
    public async Task<bool> IsStockAvailableAsync(Guid productId, int qty, ...) { ... }
    public async Task ReserveStockAsync(Guid productId, int qty, ...) { ... }
    public async Task ReleaseStockAsync(Guid productId, int qty, ...) { ... }
    // Nenhum metodo aqui trata de pagamento, cliente, endereco ou pedido
    // Alta coesao → facil de entender, testar e manter isoladamente
}

// GRASP - High Cohesion: Money
// Todos os metodos sao operacoes monetarias – nenhum trata de negocio
public sealed class Money : ValueObject
{
    public Money Add(Money other) { ... }    // soma monetaria
    public Money Subtract(Money other) { ... }    // subtracao monetaria
    public Money Multiply(decimal factor) { ... } // multiplicacao por quantidade
    public bool IsGreaterThan(Money other) { ... } // comparacao monetaria
}
```

3.6 GRASP — Controller

Padrao Controller — Objetivo: Receber e coordenar operacoes do sistema, delegando a camada de dominio sem conter logica de negocio. O Controller conhece o "o que" mas nao o "como". Separa a camada de entrada (API/CLI) da logica de dominio.

Listagem: GRASP Controller — OrderApplicationService

```
// GRASP - Controller: OrderApplicationService
// Primeiro objeto alem da API/CLI que recebe um comando do sistema.
// COORDENA sem executar logica de negocio – conhece o "o que", nao o "como".
public class OrderApplicationService
{
    private readonly OrderFactory    _orderFactory;    // SOLID - DIP: abstracoes
    private readonly OrderDomainService _orderDomainService;
    private readonly IOrderRepository _orderRepository;

    // Caso de Uso 1: Criar Pedido
```

```
public async Task<Guid> PlaceOrderAsync(
    Guid customerId, Address deliveryAddress,
    IEnumerable<(Guid ProductId, int Quantity)> items, ...)
{
    // Delega criacao a Factory – Controller nao instancia Order diretamente
    var order = await _orderFactory.CreateAsync(customerId, deliveryAddress, items);
    // Delega regra de negocio ao Domain Service – Controller nao verifica estoque
    await _orderDomainService.ConfirmOrderAsync(order);
    await _orderRepository.AddAsync(order);
    return order.Id; // retorna ID para a camada de apresentacao (API/CLI)
}

// Caso de Uso 2: Processar Pagamento
public async Task ProcessPaymentAsync(Guid orderId, string token, ...)
{
    var order = await GetOrderOrThrowAsync(orderId);
    await _orderDomainService.ProcessPaymentAsync(order, token); // delega
}

// Caso de Uso 3: Cancelar Pedido
public async Task CancelOrderAsync(Guid orderId, ...)
{
    var order = await GetOrderOrThrowAsync(orderId);
    await _orderDomainService.CancelOrderAsync(order); // delega
}
// Nenhum if/else de regra de negocio – apenas orquestracao de chamadas
}
```


4. Testes Unitarios e TDD

[Rubricas R13, R14, R15, R16] Principios F.I.R.S.T (R13); cenarios relevantes incluindo negativos (R14); mocks e stubs com Moq (R15); cobertura superior a 80% (R16).

4.1 Principios F.I.R.S.T

[Rubrica R13] F.I.R.S.T: Fast, Isolated, Repeatable, Self-verifying, Timely.

Letra	Principio	Como aplicado no projeto
F	Fast (Rapido)	Sem I/O real — repositorios InMemory, sem banco, sem rede
I	Isolated (Isolado)	Cada teste cria seus proprios objetos; sem estado compartilhado via mocks do Moq
R	Repeatable	Sem randomness, sem DateTime.Now — resultados determinísticos em qualquer maquina
S	Self-verifying	Assert.Equal(), Assert.Throws(), .Verify() — nenhuma inspecao manual necessaria
T	Timely	Testes escritos junto ao codigo de producao (TDD) — ver Order.cs e OrderTests.cs

4.2 Cenarios de Teste

[Rubrica R14] 118 cenarios de teste cobrindo todas as regras de negocio relevantes, incluindo testes positivos e negativos para cada classe do dominio (casos Theory geram multiplas execucoes xUnit por metodo).

Classe	Cenario	Tipo	Metodo de Teste
Money	Criacao com valor valido	Positivo	Constructor_ValidAmount
Money	Moeda normalizada para maiusculas	Positivo	Constructor_Normalize
Money	Arredondamento em 2 casas decimais	Positivo	Constructor_Round
Money	Soma de mesma moeda	Positivo	Add_SameCurrency
Money	Subtracao sem resultado negativo	Positivo	Subtract_SameCurrency
Money	Multiplicacao por fator positivo	Positivo	Multiply_ValidFactor
Money	Igualdade estrutural (mesmos valores)	Positivo	Equals_SameValues
Money	Valor negativo lanca ArgumentException	Negativo	Constructor_NegativeAmount
Money	Moeda invalida lanca ArgumentException	Negativo	Constructor_InvalidCurrency
Money	Soma de moedas diferentes lanca	Negativo	Add_DifferentCurrencies

Classe	Cenario	Tipo	Metodo de Teste
Money	Subtracao com resultado negativo lanca	Negativo	Subtract_ResultNegative
Order	Novo pedido inicia com status Pending	Positivo	Order_NewOrder_Status
Order	Adicionar item ao pedido	Positivo	AddItem_ValidItem
Order	Mesmo produto acumula quantidade	Positivo	AddItem_SameProduct
Order	Total calculado com multiplos produtos	Positivo	AddItem_MultipleProducts
Order	Remover item existente do pedido	Positivo	RemoveItem_Existing
Order	Confirmar pedido com itens	Positivo	Confirm_WithItems
Order	Confirm dispara OrderPlacedEvent	Positivo	Confirm_RaisesEvent
Order	Total correto no OrderPlacedEvent	Positivo	OrderPlacedEvent_Total
Order	Ciclo completo Pending ate Delivered	Positivo	FullLifecycle
Order	Cancel em Pending dispara OrderCancelledEvent	Positivo	Cancel_PendingOrder
Order	ClearDomainEvents remove todos os eventos	Positivo	ClearDomainEvents
Order	Atualizar endereco em Pending	Positivo	UpdateAddress_Pending
Order	CustomerId vazio lanca ArgumentException	Negativo	Order_EmptyCustomerId
Order	Address nulo lanca ArgumentNullException	Negativo	Order_NullAddress
Order	Confirmar sem itens lanca InvalidOperationException	Negativo	Confirm_WithNoItems
Order	AddItem apos Confirm lanca InvalidOperationException	Negativo	AddItem_AfterConfirm
Order	Transicao invalida Pending>Shipped lanca	Negativo	InvalidTransition
Order	Cancel em Delivered lanca InvalidOperationException	Negativo	Cancel_Delivered
Order	UpdateAddress apos Paid lanca	Negativo	UpdateAddress_AfterPaid
OrderStatus	Todas as 7 transicoes validas	Positivo	CanTransitionTo_Valid(7)
OrderStatus	Todas as 6 transicoes invalidas	Negativo	CanTransitionTo_Invalid(6)
OrderStatus	Igualdade entre instancias	Positivo	Equality_SameStatus
Product	Produto com estoque disponivel	Positivo	Product_WithStock
Product	Reservar estoque valido	Positivo	ReserveStock_Valid
Product	Reservar todo o estoque fica unavailable	Positivo	ReserveStock_AllUnits
Product	Repor estoque aumenta e fica available	Positivo	ReplenishStock

Classe	Cenario	Tipo	Metodo de Teste
Product	Desativar produto	Positivo	Deactivate
Product	Reserva acima do disponivel lanca	Negativo	ReserveStock_MoreThan
Product	Reservar quantidade zero lanca	Negativo	ReserveStock_Zero
Product	Repor quantidade zero lanca	Negativo	ReplenishStock_Zero
Customer	Cliente adulto valido criado	Positivo	Customer_ValidData
Customer	Atualizar e-mail	Positivo	UpdateEmail_Valid
Customer	E-mail normalizado para lowercase	Positivo	Email_Normalize
Customer	CPF valido armazena apenas digitos	Positivo	Cpf_Valid
Customer	Nome vazio lanca ArgumentException	Negativo	Customer_EmptyName
Customer	Menor de 18 anos lanca InvalidOperation	Negativo	Customer_Under18
Customer	E-mail invalido lanca ArgumentException	Negativo	Email_Invalid(3)
Customer	CPF invalido lanca ArgumentException	Negativo	Cpf_Invalid
OrderDomainSvc	Confirmar com estoque OK muda status	Positivo	ConfirmAsync_StockOK
OrderDomainSvc	Pagamento bem-sucedido marca como Paid	Positivo	ProcessPayment_Success
OrderDomainSvc	Cancelar Pending nao libera estoque	Positivo	CancelAsync_Pending
OrderDomainSvc	Cancelar pedido pago libera estoque	Positivo	CancelAsync_Paid_ReleasesStock
OrderDomainSvc	Confirmar sem estoque lanca	Negativo	ConfirmAsync_NoStock
OrderDomainSvc	Pagamento falho nao muda status	Negativo	ProcessPayment_Failed
OrderFactory	Cliente e produto validos cria pedido	Positivo	CreateAsync_ValidCustomerProduct
OrderFactory	Cliente inexistente lanca	Negativo	CreateAsync_CustomerNotFound
OrderFactory	Cliente inativo lanca	Negativo	CreateAsync_InactiveCustomer
OrderFactory	Produto inexistente lanca	Negativo	CreateAsync_ProductNotFound
OrderFactory	Produto indisponivel lanca	Negativo	CreateAsync_ProductUnavailable
OrderFactory	Dependencia nula no construtor lanca	Negativo	Constructor_NullDeps
InventoryDomainSvc	Estoque suficiente retorna true	Positivo	IsStockAvailable_Sufficient
InventoryDomainSvc	Reservar estoque reduz e persiste	Positivo	ReserveStock_Existing

Classe	Cenario	Tipo	Metodo de Teste
InventoryDomainSvc	Liberar estoque repoe e persiste	Positivo	ReleaseStock_Existing
InventoryDomainSvc	Estoque insuficiente retorna false	Negativo	IsStockAvailable_Insufficient
InventoryDomainSvc	Produto inexistente retorna false	Negativo	IsStockAvailable_ProductNull
InventoryDomainSvc	Reservar produto inexistente lanca	Negativo	ReserveStock_NotFound
InventoryDomainSvc	Liberar produto inexistente lanca	Negativo	ReleaseStock_NotFound
InventoryDomainSvc	Repositorio nulo no construtor lanca	Negativo	Constructor_NullRepo
Address	Criacao com dados validos	Positivo	Address_ValidData
Address	Igualdade por valor	Positivo	Address_Equality
Address	Campo obrigatorio vazio lanca	Negativo	Address_EmptyField(3)

Total: 118 execucoes xUnit — cenarios positivos e negativos — distribuidas em 9 classes de teste (metodos [Fact] + casos [Theory])

4.3 Mocks e Stubs com Moq

[Rubrica R15] O framework **Moq** cria doubles de teste garantindo isolamento total. **Mock**: verificado com `Verify()`. **Stub**: retorna valores pre-configurados.

Listagem: OrderDomainServiceTests.cs — Mocks e Stubs com Moq

```
public class OrderDomainServiceTests
{
    // STUB: simula repositorio – nunca toca banco real (FAST + ISOLATED)
    private readonly Mock<IOrderRepository> _orderRepoMock = new();
    private readonly Mock<IProductRepository> _productRepoMock = new();
    // MOCK: gateway de pagamento – comportamento VERIFICADO com .Verify()
    private readonly Mock<IPaymentService> _paymentServiceMock = new();
    private readonly Mock<IInventoryDomainService> _inventoryMock = new();

    [Fact]
    public async Task ProcessPaymentAsync_SuccessfulPayment_ShouldMarkAsPaid()
    {
        // ARRANGE – STUB do gateway retorna sucesso (pre-configurado)
        _paymentServiceMock
            .Setup(p => p.ChargeAsync(
                It.IsAny<Guid>(), It.IsAny<Money>(), "valid_token", default))
            .ReturnsAsync(PaymentResult.Success("ch_abc123")); // stub

        _inventoryMock
            .Setup(s => s.ReserveStockAsync(It.IsAny<Guid>(), It.IsAny<int>(), default))
            .Returns(Task.CompletedTask);

        var order = MakeOrderWithItem();
        order.Confirm(); // pre-condicao: status Confirmed

        // ACT
        await _sut.ProcessPaymentAsync(order, "valid_token");

        // ASSERT – verifica estado do DOMINIO (nao do mock)
        Assert.Equal(OrderStatus.Paid, order.Status);

        // VERIFY – MOCK verifica que o gateway foi chamado 1 vez com os args corretos
        _paymentServiceMock.Verify(p =>
```

```

        p.ChargeAsync(order.Id, It.IsAny<Money>(), "valid_token", default),
        Times.Once); // garantia de que a cobrança foi feita
    }

    [Fact]
    public async Task ConfirmOrderAsync_InsufficientStock_ShouldThrow() // TESTE NEGATIVO
    {
        // ARRANGE – STUB retorna false (sem estoque)
        _inventoryMock
            .Setup(s => s.IsStockAvailableAsync(It.IsAny<Guid>(), It.IsAny<int>(), default))
            .ReturnsAsync(false);

        var order = MakeOrderWithItem(); // pedido em status Pending

        // ACT + ASSERT – regra de negocio: deve lancar excecao
        await Assert.ThrowsAsync<InvalidOperationException>(() =>
            _sut.ConfirmOrderAsync(order));

        // Order permanece Pending – estado nao mudou
        Assert.Equal(OrderStatus.Pending, order.Status);
    }
}

```

4.4 Relatório de Cobertura de Testes

[Rubrica R16] Relatório gerado com **coverlet.msbuild** e **ReportGenerator**. Cobertura de 89,3% supera a meta de 80%.

Classe / Arquivo	Linhas	Cobertas	Branches	Cobertos	Cobertura
Common/Entity.cs	20	10	16	3	50%
Common/ValueObject.cs	16	10	14	6	62%
Common/AggregateRoot.cs	9	9	0	0	100%
Common/DomainEvent.cs	2	2	0	0	100%
OrderManagement/Order.cs	77	76	24	23	99%
OrderManagement/OrderItem.cs	25	19	10	5	76%
OrderManagement/Money.cs	47	43	12	12	91%
OrderManagement/Address.cs	37	37	6	5	100%
OrderManagement/OrderStatus.cs	26	24	34	30	92%
OrderManagement/OrderDomainService.cs	48	48	24	19	100%
OrderManagement/OrderFactory.cs	24	24	14	14	100%
OrderManagement/Events/*.cs	16	12	0	0	75%
Inventory/Product.cs	40	39	18	15	98%
Inventory/ProductCode.cs	16	10	10	8	62%
Inventory/InventoryDomainService.cs	21	21	10	10	100%
Customer/Customer.cs	33	30	14	11	91%
Customer/Email.cs	16	15	4	4	94%
Customer/Cpf.cs	30	25	24	18	83%

Classe / Arquivo	Linhas	Coberturas	Branches	Cobertos	Cobertura
Payment/PaymentResult.cs	18	11	4	0	61%
TOTAL — Código de Domínio	521	465	238	183	89,3%

Cobertura de Linhas do Domínio:	89,3% (465/521)
Cobertura de Branches (Desvios):	76,9% (183/238)
Cobertura de Metodos do Domínio:	81,0%
Meta de 80% de cobertura:	SUPERADA

Listagem: Output do terminal — dotnet test com coverlet

```
$ dotnet test /p:CollectCoverage=true /p:CoverletOutputFormat=cobertura \
/p:Include="[Domain]*"
```

```
Build succeeded.
```

```
Domain.Tests -> bin/Debug/net10.0/Domain.Tests.dll
```

```
Test run for Domain.Tests.dll (.NETCoreApp,Version=v10.0)
```

```
Microsoft (R) Test Execution Command Line Tool Version 18.0.1
```

```
Passed! - Failed: 0, Passed: 118, Skipped: 0, Total: 118
```

```
Calculating coverage result...
```

```
+-----+-----+-----+-----+
| Module | Line  | Branch | Method |
+-----+-----+-----+-----+
| Domain | 89.25% | 76.89% | 81.02% |
+-----+-----+-----+-----+
```

5. Conclusão

O projeto **Sistema de Gestão de Pedidos de E-commerce** demonstrou na prática a aplicação integrada de todos os conceitos da disciplina Tecnologia .NET:

- **DDD:** 4 Bounded Contexts com Aggregates que garantem invariantes, Domain Services que coordenam operações e Anti-Corruption Layer isolando o domínio do gateway externo.
- **OO em C#:** Hierarquia de 3 níveis (Entity > AggregateRoot > concreto); encapsulamento via private/internal/protected; polimorfismo via sobrescrita de métodos abstratos.
- **SOLID e GRASP:** SRP em cada classe; OCP via herança de abstratos; DIP via interfaces e DI; Low Coupling por repositórios e IDs; High Cohesion em cada serviço; Controller via OrderApplicationService.
- **Testes Unitários:** 118 execuções de teste (positivos e negativos), isolamento via Moq, F.I.R.S.T respeitados e cobertura de 89,3% — superando a meta de 80%.

Repositorio Publico:	https://github.com/gabrielbller/OrderFlow
Estrutura:	src/Domain/ src/Infrastructure/ src/Application/ tests/Domain.Tests/
Tecnologias:	.NET 10.0, C# 14, xUnit 2.7, Moq 4.20, coverlet.msbuild 6.0